

Búsqueda Binaria (C++)

Generated by Doxygen 1.16.1

1 Búsqueda Binaria (C++)	1
1.1 Características	1
1.2 Estructura del proyecto	1
1.3 Documentación	2
1.4 Ejecución	2
1.5 Autora	2
1.6 Maestro	2
1.7 Materia y Programa Académico	2
1.8 Fecha	2
2 Análisis	3
2.1 Cuestiones	3
2.2 Análisis comparativo: Métodos de ordenamiento	3
2.3 Complejidad algorítmica: Espacial y temporal	4
2.4 Casos de uso: Binaria o línea	4
3 Pruebas	5
3.1 Resultados de Pruebas	5
4 Bibliografía	9
5 Directory Hierarchy	11
5.1 Directories	11
6 Class Index	13
6.1 Class List	13
7 File Index	15
7.1 File List	15
8 Directory Documentation	17
8.1 headers Directory Reference	17
8.2 src Directory Reference	17
9 Class Documentation	19
9.1 BusquedaBin< T > Class Template Reference	19
9.1.1 Detailed Description	19
9.1.2 Constructor & Destructor Documentation	19
9.1.2.1 BusquedaBin()	19
9.1.3 Member Function Documentation	20
9.1.3.1 buscalterativa()	20
9.1.3.2 buscaRecursiva()	21
10 File Documentation	23
10.1 headers/busqueda_bin.h File Reference	23
10.1.1 Detailed Description	23

10.2 busqueda_bin.h	23
10.3 README.md File Reference	24
10.4 src/busqueda_bin.cpp File Reference	24
10.4.1 Detailed Description	24
10.5 busqueda_bin.cpp	24
10.6 src/main.cpp File Reference	25
10.6.1 Detailed Description	25
10.6.2 Function Documentation	26
10.6.2.1 main()	26
10.6.2.2 menu()	27
10.7 main.cpp	27
Index	31

Chapter 1

Búsqueda Binaria (C++)

Implementación de una clase Grafo para modelar redes de datos y encontrar los caminos más cortos entre sus vertices, sea el grafo dirigido o no dirigido, haciendo uso del algoritmo de Dijkstra.

1.1 Características

- Menú interactivo.
- Inicialización de vector con n elementos.
- Inserción de elementos en el vector.
- Visualización de la búsqueda de un número "x" mediante la versión iterativa del algoritmo.
- Visualización de la búsqueda de un número "x" mediante la versión recursiva del algoritmo.

1.2 Estructura del proyecto

- doc/
 - html/
 - latex/
 - images/
 - * Imágenes de resultados de pruebas.
- src/
 - [busqueda_bin.cpp](#)
 - [main.cpp](#)
- headers/
 - [busqueda_bin.h](#)

1.3 Documentación

Toda la documentación ha sido generada con Doxygen.

Para visualizar: Diríjase a `doc/html/index.html` Posteriormente, abra con su navegador de preferencia.

Para visualizar todos los comentarios: Vaya a "Class Members", después seleccione "All".

O abra el archivo PDF "refman.pdf".

1.4 Ejecución

- Usando comandos: Abra su terminal, y ejecute el siguiente código...

```
g++ src/main.cpp -Iheaders -o busqBin ./busqBin
```

1.5 Autora

Camila Andrea Jaimes Martinez.

1.6 Maestro

Manuel Balderas Victoria.

1.7 Materia y Programa Académico

- Estructura de Datos (Aula).
- Ingeniería en Software y Minería de Datos (1er año).

1.8 Fecha

Martes 12 de Abril, 2026.

Chapter 2

Análisis

Reflexión teórica y Análisis comparativo del programa.

2.1 Cuestiones

En este apartado se responden las siguientes cuestiones...

- 1) Comparación entre los distintos métodos de ordenamiento vistos en clase.
- 2) Complejidad algorítmica (O) de la búsqueda binaria.
- 3) Casos de uso: ¿Cuándo es preferible usar búsqueda binaria sobre búsqueda lineal?

2.2 Análisis comparativo: Métodos de ordenamiento

1) En clase se discutieron diversos métodos de ordenamiento, entre estos los métodos iterativos como los algoritmos de ordenamiento de la burbuja, de selección, etcétera, además de métodos recursivos como el algoritmo de Quicksort y Mergesort.

Los métodos iterativos son ideales para estructuras de datos con tamaños desde pequeños a medianos debido al uso de ciclos while en sus algoritmos que les permite consumir memoria constante (misma cantidad de RAM por ejecución), sin embargo, esta misma característica los convierte en métodos ineficientes en el caso de grandes estructuras de datos en donde la cantidad de operaciones a realizar puede ser inmensa. Por ejemplo, el algoritmo de la burbuja utiliza su ciclo while para comparar elementos adyacentes de su estructura de datos hasta alcanzar el orden correcto.

- La complejidad algorítmica de estos métodos suele ser de $O(n^2)$.

Por otra parte, los métodos recursivos son óptimos para estructuras de datos con tamaños desde pequeños a grandes; su recursividad les permite dividir el espacio de búsqueda fácilmente en subrangos y posteriormente ordenar los elementos de la estructura de datos en forma ascendente. Por ejemplo, el algoritmo de Quicksort divide la estructura de datos mediante la selección de un pivote y ordena los elementos menores a la derecha, mientras que coloca los elementos mayores a su derecha.

- La complejidad algorítmica de estos métodos suele ser de $O(n \log n)$.

2.3 Complejidad algorítmica: Espacial y temporal

2) Las complejidades son...

- Búsqueda binaria de forma iterativa.

Su complejidad algorítmica promedio se categoriza como $O(\log n)$ en cuestión de tiempo y $O(1)$ en cuestión de memoria, pues crece de forma logarítmica el tiempo de ejecución con respecto al tamaño del vector, y además, usará la misma cantidad de memoria en RAM por ejecución. En el mejor de los casos, su complejidad algorítmica temporal será $O(1)$, lo cual es posible cuando el elemento "x" a buscar se encuentra en el medio del vector, mientras que su complejidad espacial permanecerá como $O(1)$ al consumir memoria constante.

- Búsqueda binaria de forma recursiva.

Su complejidad algorítmica promedio se categoriza como $O(\log n)$ en cuestión de tiempo y $O(\log n)$ en cuestión de memoria, pues tanto el tiempo como la memoria utilizada en RAM crece proporcionalmente al logaritmo del tamaño del vector: esto se puede comprobar al tener un vector de gran tamaño, donde la cantidad de divisiones aumenta considerablemente y las llamadas a la misma función se quedan congeladas, utilizando aún más memoria. En el mejor de los casos, su complejidad algorítmica temporal será $O(1)$, lo cual es posible cuando el elemento "x" a buscar se encuentra en el medio del vector, y complejidad espacial se convierte en $O(1)$ al consumir memoria constante.

2.4 Casos de uso: Binaria o lineal

3) Es mejor aplicar el algoritmo de búsqueda binaria en casos donde el espacio de búsqueda tiene demasiados ejemplos, por ejemplo, para un vector de 15,000 elementos, los cuales deben ser ingresados de forma ascendente o ser organizados con una única procedimiento para evitar mayor gasto de memoria en RAM. Dada su complejidad temporal de $O(\log n)$, se puede saber la cantidad de operaciones que realizará aplicando un logaritmo base 2 sobre la cantidad de elementos que hay en el espacio de búsqueda: si se sigue el ejemplo, para un vector de 15,000 elementos, la cantidad de operaciones a realizar se aproxima a 13.

Si fuera un espacio de búsqueda pequeña, por ejemplo, de 10 elementos, entonces es más susceptible la búsqueda lineal, dada tiene complejidad espacial de $O(1)$, donde la memoria en RAM que se utiliza por ejecución es constante. Sin embargo, su complejidad temporal es de $O(n)$, lo que implica que el tiempo por ejecución aumenta exactamente igual al tamaño de datos y lo hace un algoritmo ineficiente para grandes cantidades de datos.

Chapter 3

Pruebas

Ejecución del programa.

3.1 Resultados de Pruebas

Las siguientes imágenes muestran la ejecución exitosa del programa correspondiente al algoritmo de Búsqueda Binaria, el cual tiene las funciones de...

- Inicialización de vector con n elementos.
- Inserción de elementos en el vector.
- Visualización de la búsqueda de un número "x" mediante la versión iterativa del algoritmo.
- Visualización de la búsqueda de un número "x" mediante la versión recursiva del algoritmo.

El primer caso corresponde a un vector cuyos elementos fueron ingresados, en forma ascendente, por el usuario. A continuación, se muestra la ejecución del programa para este grafo.

```

[camila@MacBook-Air-de-Camila busq_bin % ./busqBin

-----| MENÚ ! -----
-----> ¡Búsqueda binaria!
1) Crear un vector de n elementos.
2) Buscar un número a partir de la versión iterativa del algoritmo.
3) Buscar un número a partir de la versión recursiva del algoritmo.
4) Salir del programa.

--> Seleccione una opción: 1

¿De cuántos números desea que sea el vector a utilizar? 6

----> ¿Qué números desea agregar al vector?
[Índice 0] Número: 2
[Índice 1] Número: 4
[Índice 2] Número: 7
[Índice 3] Número: 19
[Índice 4] Número: 33
[Índice 5] Número: 99

-----| MENÚ ! -----
-----> ¡Búsqueda binaria!
1) Crear un vector de n elementos.
2) Buscar un número a partir de la versión iterativa del algoritmo.
3) Buscar un número a partir de la versión recursiva del algoritmo.
4) Salir del programa.

--> Seleccione una opción: 2

Ingrese el número a buscar mediante la versión iterativa: 4

¡Se ha encontrado el número!
----> Índice del número 4 : 1

-----| MENÚ ! -----
-----> ¡Búsqueda binaria!
1) Crear un vector de n elementos.
2) Buscar un número a partir de la versión iterativa del algoritmo.
3) Buscar un número a partir de la versión recursiva del algoritmo.
4) Salir del programa.

--> Seleccione una opción: 2

Ingrese el número a buscar mediante la versión iterativa: 7

¡Se ha encontrado el número!
----> Índice del número 7 : 2

-----| MENÚ ! -----
-----> ¡Búsqueda binaria!
1) Crear un vector de n elementos.
2) Buscar un número a partir de la versión iterativa del algoritmo.
3) Buscar un número a partir de la versión recursiva del algoritmo.
4) Salir del programa.

--> Seleccione una opción: 3

```

Figure 3.1 Ejecución del programa en un vector inicialmente ordenado (Parte 1)

```

--> Seleccione una opción: 2

Ingrese el número a buscar mediante la versión iterativa: 4

¡Se ha encontrado el número!
--> Índice del número 4 : 1

-----
_ _ _ _ _ ; MENÚ ! _ _ _ _ _

-----> ¡Búsqueda binaria!
1) Crear un vector de n elementos.
2) Buscar un número a partir de la versión iterativa del algoritmo.
3) Buscar un número a partir de la versión recursiva del algoritmo.
4) Salir del programa.

--> Seleccione una opción: 2

Ingrese el número a buscar mediante la versión iterativa: 7

¡Se ha encontrado el número!
--> Índice del número 7 : 2

-----
_ _ _ _ _ ; MENÚ ! _ _ _ _ _

-----> ¡Búsqueda binaria!
1) Crear un vector de n elementos.
2) Buscar un número a partir de la versión iterativa del algoritmo.
3) Buscar un número a partir de la versión recursiva del algoritmo.
4) Salir del programa.

--> Seleccione una opción: 3

Ingrese el número a buscar mediante la versión recursiva: 19

¡Se ha encontrado el número!
--> Índice del número 19 : 3

-----
_ _ _ _ _ ; MENÚ ! _ _ _ _ _

-----> ¡Búsqueda binaria!
1) Crear un vector de n elementos.
2) Buscar un número a partir de la versión iterativa del algoritmo.
3) Buscar un número a partir de la versión recursiva del algoritmo.
4) Salir del programa.

--> Seleccione una opción: 3

Ingrese el número a buscar mediante la versión recursiva: 8

No se ha encontrado el número en el vector.

-----
_ _ _ _ _ ; MENÚ ! _ _ _ _ _

-----> ¡Búsqueda binaria!
1) Crear un vector de n elementos.
2) Buscar un número a partir de la versión iterativa del algoritmo.
3) Buscar un número a partir de la versión recursiva del algoritmo.
4) Salir del programa.

--> Seleccione una opción: 4

camila@MacBook-Air-de-Camila busq_bin %

```

Figure 3.2 Ejecución del programa en un vector inicialmente ordenado (Part 2 - Final)

El segundo caso corresponde a un vector cuyos elementos fueron ingresados, sin orden alguno, por el usuario. Para ello, se implementa la función `sort()` de la librería `<algorithm>`, de tal manera que los elementos del vector sean organizados en forma ascendente correctamente, lo cual sujeta a los elementos del vector a un cambio de índices. A continuación, se muestra la ejecución del programa para este grafo.

```

_ _ _ _ _ ; MENÚ ! _ _ _ _ _
-----> ¡Búsqueda binaria!
1) Crear un vector de n elementos.
2) Buscar un número a partir de la versión iterativa del algoritmo.
3) Buscar un número a partir de la versión recursiva del algoritmo.
4) Salir del programa.

--> Seleccione una opción: 1

¿De cuántos números desea que sea el vector a utilizar? 7

----> ¿Qué números desea agregar al vector?
[Índice 0] Número: 99
[Índice 1] Número: 2
[Índice 2] Número: 44
[Índice 3] Número: 56
[Índice 4] Número: 12
[Índice 5] Número: 60
[Índice 6] Número: 34

_ _ _ _ _ ; MENÚ ! _ _ _ _ _
-----> ¡Búsqueda binaria!
1) Crear un vector de n elementos.
2) Buscar un número a partir de la versión iterativa del algoritmo.
3) Buscar un número a partir de la versión recursiva del algoritmo.
4) Salir del programa.

--> Seleccione una opción: 2

Ingrese el número a buscar mediante la versión iterativa: 60

¡Se ha encontrado el número!
--> Índice del número 60 : 5

_ _ _ _ _ ; MENÚ ! _ _ _ _ _
-----> ¡Búsqueda binaria!
1) Crear un vector de n elementos.
2) Buscar un número a partir de la versión iterativa del algoritmo.
3) Buscar un número a partir de la versión recursiva del algoritmo.
4) Salir del programa.

--> Seleccione una opción: 3

Ingrese el número a buscar mediante la versión recursiva: 12

¡Se ha encontrado el número!
--> Índice del número 12 : 1

_ _ _ _ _ ; MENÚ ! _ _ _ _ _
-----> ¡Búsqueda binaria!
1) Crear un vector de n elementos.
2) Buscar un número a partir de la versión iterativa del algoritmo.
3) Buscar un número a partir de la versión recursiva del algoritmo.
4) Salir del programa.

--> Seleccione una opción: 4

camila@MacBook-Air-de-Camila busq_bin %

```

Figure 3.3 Ejecución del programa en un vector inicialmente no ordenado (Parte 1 - Final)

Chapter 4

Bibliografía

Fuentes consultadas.

Las fuentes consultadas (en formato APA) para la realización de este programa fueron...

- 1) Binary search. (2014, enero 28). GeeksforGeeks. <https://www.geeksforgeeks.org/dsa/binary-search/>
- 2) DSA binary search. (s/f). W3schools.com. Recuperado el 12 de mayo de 2026, de https://www.w3schools.com/dsa/dsa_algo_binarysearch.php
- 3) C++ algorithm Library Reference (algorithm functions). (s/f). W3schools.com. Recuperado el 12 de mayo de 2026, de https://www.w3schools.com/cpp/cpp_ref_algorithm.asp

Chapter 5

Directory Hierarchy

5.1 Directories

headers	17
busqueda_bin.h	23
src	17
busqueda_bin.cpp	24
main.cpp	25

Chapter 6

Class Index

6.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

BusquedaBin< T >	19
--	----

Chapter 7

File Index

7.1 File List

Here is a list of all files with brief descriptions:

headers/ busqueda_bin.h	
Se define la estructura de datos para la implementación del algoritmo de búsqueda binaria . .	23
src/ busqueda_bin.cpp	
Implementación de de los constructores y métodos de la plantilla BúsquedaBin	24
src/ main.cpp	
Busqueda binaria	25

Chapter 8

Directory Documentation

8.1 headers Directory Reference

Files

- file [busqueda_bin.h](#)

Se define la estructura de datos para la implementación del algoritmo de búsqueda binaria.

8.2 src Directory Reference

Files

- file [busqueda_bin.cpp](#)

Implementación de de los constructores y métodos de la plantilla BúsquedaBin.

- file [main.cpp](#)

busqueda binaria

Chapter 9

Class Documentation

9.1 BusquedaBin< T > Class Template Reference

```
#include <busqueda_bin.h>
```

Public Member Functions

- [BusquedaBin](#) (const vector< T > &vec_datos)
Inicializa un vector, o espacio de búsqueda, de n elementos.
- int [buscalterativa](#) (const T &x) const
Realiza la búsqueda binaria de forma iterativa en todo el vector.
- int [buscaRecursiva](#) (const T &x) const
Recibe el elemento a buscar para la búsqueda binaria de forma recursiva.

9.1.1 Detailed Description

```
template<typename T>  
class BusquedaBin< T >
```

Definition at line 15 of file [busqueda_bin.h](#).

9.1.2 Constructor & Destructor Documentation

9.1.2.1 BusquedaBin()

```
template<typename T>  
BusquedaBin< T >::BusquedaBin (  
    const vector< T > & vec_datos)
```

Inicializa un vector, o espacio de búsqueda, de n elementos.

Copia los datos recibidos del vector obtenido del usuario en el vector, o espacio de búsqueda, interno.

Parameters

<code>vec_datos</code>	Apuntador al vector obtenido del usuario.
------------------------	---

Definition at line 60 of file [busqueda_bin.cpp](#).

```
00061 {
00062     estruct_datos = vec_datos;
00063 }
```

9.1.3 Member Function Documentation**9.1.3.1 buscalterativa()**

```
template<typename T>
int BusquedaBin< T >::buscaIterativa (
    const T & x) const
```

Realiza la búsqueda binaria de forma iterativa en todo el vector.

Primero, delimita el espacio de búsqueda en el primer (low) y último (high) elemento del vector, y después lo divide en dos mitades usando la fórmula "low + (high - low) / 2" para encontrar el índice central sin riesgo de que se presente un desbordamiento (overflow). Posteriormente, pregunta si el número en el índice central es igual al número "x" que se busca.

Si es mayor, se dirige al subrango derecho, y si es menor, al izquierdo. Nuevamente, encuentra la mitad del subrango, y repite el proceso de búsqueda binaria.

Parameters

<code>x</code>	Elemento en el vector a buscar.
----------------	---------------------------------

Definition at line 81 of file [busqueda_bin.cpp](#).

```
00082 {
00083     int low = 0;
00084     int high = estruct_datos.size() - 1;
00085     while (low <= high)
00086     {
00087         int mid = low + (high - low) / 2;
00088         if (estruct_datos[mid] == x)
00089         {
00090             return mid;
00091         }
00092         else if (estruct_datos[mid] < x)
00093         {
00094             low = mid + 1;
00095         }
00096         else
00097         {
00098             high = mid - 1;
00099         }
00100     }
00101     return -1;
00102 }
00103
00104
00105 }
```


9.1.3.2 buscaRekursiva()

```
template<typename T>
int BusquedaBin< T >::buscaRekursiva (
    const T & x) const
```

Recibe el elemento a buscar para la búsqueda binaria de forma recursiva.

A partir del número obtenido del usuario, se utiliza la función privada de búsqueda binaria recursiva para calcular, automáticamente, los índices de low y high.

Parameters

<i>x</i>	Elemento en el vector a buscar.
----------	---------------------------------

Definition at line 117 of file [busqueda_bin.cpp](#).

```
00118 {
00119     return buscaRekursiva(x, 0, estruct_datos.size() - 1);
00120 }
```

The documentation for this class was generated from the following files:

- [headers/busqueda_bin.h](#)
- [src/busqueda_bin.cpp](#)

Chapter 10

File Documentation

10.1 headers/busqueda_bin.h File Reference

Se define la estructura de datos para la implementación del algoritmo de búsqueda binaria.

```
#include <vector>
#include "../src/busqueda_bin.cpp"
```

Classes

- class [BusquedaBin< T >](#)

10.1.1 Detailed Description

Se define la estructura de datos para la implementación del algoritmo de búsqueda binaria.

Author

Camila Andrea Jaimes Martínez.

a

Definition in file [busqueda_bin.h](#).

10.2 busqueda_bin.h

[Go to the documentation of this file.](#)

```
00001 #pragma once
00002 #include <vector>
00003
00004 using namespace std;
00005
00013
00014 template <typename T>
00015 class BusquedaBin
00016 {
00017     private:
00018         vector<T> estruct_datos;
00019
00020         int busquedaRekursiva(const T& x, int low, int high) const;
00021     public:
00022         BusquedaBin(const vector<T>& vec_datos);
00023
00024         int buscaIterativa(const T& x) const;
00025         int buscaRekursiva(const T& x) const;
00026 };
00027
00028
00029 #include "../src/busqueda_bin.cpp"
```

10.3 README.md File Reference

10.4 src/busqueda_bin.cpp File Reference

Implementación de de los constructores y métodos de la plantilla BúsquedaBin.

```
#include <iostream>
#include "busqueda_bin.h"
```

10.4.1 Detailed Description

Implementación de de los constructores y métodos de la plantilla BúsquedaBin.

Definition in file [busqueda_bin.cpp](#).

10.5 busqueda_bin.cpp

[Go to the documentation of this file.](#)

```
00001 #include <iostream>
00002 #include "busqueda_bin.h"
00003
00008
00009 using namespace std;
00010
00011 /*****
00012
00026
00027 template <typename T>
00028 int BusquedaBin<T>::busquedaRecursiva(const T& x, int low, int high) const
00029 {
00030     if (low > high)
00031     {
00032         return -1;
00033     }
00034
00035     int mid = low + (high - low) / 2;
00036
00037     if (estruct_datos[mid] == x)
00038     {
00039         return mid;
00040     }
00041     else if (estruct_datos[mid] < x)
00042     {
00043         return busquedaRecursiva(x, mid + 1, high);
00044     }
00045     else
00046     {
00047         return busquedaRecursiva(x, low, mid - 1);
00048     }
00049 }
00050
00051 /*****
00052
00058
00059 template <typename T>
00060 BusquedaBin<T>::BusquedaBin(const vector<T>& vec_datos)
00061 {
00062     estruct_datos = vec_datos;
00063 }
00064
00065 /*****
00066
00079
00080 template <typename T>
00081 int BusquedaBin<T>::buscaIterativa(const T& x) const
00082 {
00083     int low = 0;
```

```

00084     int high = estruct_datos.size() - 1;
00085
00086     while (low <= high)
00087     {
00088         int mid = low + (high - low) / 2;
00089
00090         if (estruct_datos[mid] == x)
00091         {
00092             return mid;
00093         }
00094         else if (estruct_datos[mid] < x)
00095         {
00096             low = mid + 1;
00097         }
00098         else
00099         {
00100             high = mid - 1;
00101         }
00102     }
00103
00104     return -1;
00105 }
00106
00107 /*****
00108
00115
00116 template <typename T>
00117 int BusquedaBin<T>::buscaRekursiva(const T& x) const
00118 {
00119     return busquedaRekursiva(x, 0, estruct_datos.size() - 1);
00120 }
00121
00122 *****/

```

10.6 src/main.cpp File Reference

busqueda binaria

```

#include <iostream>
#include <algorithm>
#include "busqueda_bin.h"

```

Functions

- int [menu](#) ()
Menú del programa que maneja el input (aportación) del usuario.
- int [main](#) ()

10.6.1 Detailed Description

busqueda binaria

Las opciones son las siguientes:

- 1) Crear un vector con tamaño de n elementos.
- 2) Buscar un número a partir de la versión iterativa del algoritmo de búsqueda binaria.
- 3) Buscar un número a partir de la versión recursiva del algoritmo de búsqueda binaria.
- 4) Salir del programa.

Definition in file [main.cpp](#).

10.6.2 Function Documentation

10.6.2.1 main()

```
int main ()
```

- < Puntero a la instancia de la clase [BusquedaBin](#).
- < Número de elementos en el vector ordenado a utilizar.
- < Ordena automáticamente el vector en forma ascendente.
- < Elimina el vector anterior si fue creado previamente.
- < Número a buscar en el vector ordenado.
- < Número a buscar en el vector ordenado.
- < Libera la memoria del vector antes de salir del programa.

Definition at line 20 of file [main.cpp](#).

```
00021 {
00022     BusquedaBin<int>* busqueda = nullptr;
00023
00024     while (true)
00025     {
00026         switch (menu())
00027         {
00028             case 1:
00029             {
00030                 int tam;
00031                 cout << "\n¿De cuántos números desea que sea el vector a utilizar? ";
00032                 cin >> tam;
00033
00034                 vector<int> elementos(tam);
00035
00036                 cout << "\n---> ¿Qué números desea agregar al vector?\n";
00037
00038                 for (int i = 0; i < tam; i++)
00039                 {
00040                     cout << "[Índice " << i << "] " << "Número: ";
00041                     cin >> elementos[i];
00042                 }
00043
00044                 sort(elementos.begin(), elementos.end());
00045
00046                 if (busqueda != nullptr)
00047                 {
00048                     delete busqueda;
00049                 }
00050
00051                 busqueda = new BusquedaBin<int>(elementos);
00052                 break;
00053             }
00054             case 2:
00055             {
00056                 if (busqueda == nullptr)
00057                 {
00058                     cout << "\nNo se ha creado un vector aún.";
00059                     break;
00060                 };
00061
00062                 int num;
00063                 cout << "\nIngrese el número a buscar mediante la versión iterativa: ";
00064                 cin >> num;
00065
00066                 int indice = busqueda->buscaIterativa(num);
00067
00068                 if (indice == -1)
00069                 {
00070                     cout << "\nNo se ha encontrado el número en el vector.";
00071                 }
00072                 else
00073                 {
00074                     cout << "\n;Se ha encontrado el número!";
00075                     cout << "\n---> Índice del número " << num << " : " << indice;
```

```

00076         }
00077         break;
00078     }
00079     case 3:
00080     {
00081         if (busqueda == nullptr)
00082         {
00083             cout << "\nNo se ha creado un vector aún.";
00084             break;
00085         };
00086
00087         int num;
00088         cout << "\nIngrese el número a buscar mediante la versión recursiva: ";
00089         cin >> num;
00090
00091         int indice = busqueda->buscaRecursiva(num);
00092
00093         if (indice == -1)
00094         {
00095             cout << "\nNo se ha encontrado el número en el vector.";
00096         }
00097         else
00098         {
00099             cout << "\n;Se ha encontrado el número!";
00100             cout << "\n--> Índice del número " << num << " : " << indice;
00101         }
00102         break;
00103     }
00104     case 4:
00105     {
00106         cout << "\nSaliendo del programa...";
00107         exit(0);
00108         break;
00109     }
00110     }
00111 }
00112
00113 delete busqueda;
00114 }

```

10.6.2.2 menu()

```
int menu ()
```

Menú del programa que maneja el input (aportación) del usuario.

Devuelve una opción a la función [main\(\)](#).

Definition at line 124 of file [main.cpp](#).

```

00125 {
00126     int opcion;
00127     do
00128     {
00129         cout << "\n\n----- ; MENÚ ! -----";
00130         cout << "\n\n-----> ;Búsqueda binaria!\n";
00131         cout << "1) Crear un vector de n elementos.\n";
00132         cout << "2) Buscar un número a partir de la versión iterativa del algoritmo.\n";
00133         cout << "3) Buscar un número a partir de la versión recursiva del algoritmo.\n";
00134         cout << "4) Salir del programa.\n";
00135         cout << "\n--> Seleccione una opción: ";
00136         cin >> opcion;
00137     } while (opcion < 1 || opcion > 4);
00138     return opcion;
00139 }

```

10.7 main.cpp

[Go to the documentation of this file.](#)

```

00001 #include <iostream>
00002 #include <algorithm>
00003 #include "busqueda_bin.h"
00004
00015

```

```

00016 int menu();
00017
00018 using namespace std;
00019
00020 int main()
00021 {
00022     BusquedaBin<int>* busqueda = nullptr;
00023
00024     while (true)
00025     {
00026         switch (menu())
00027         {
00028             case 1:
00029             {
00030                 int tam;
00031                 cout << "\n¿De cuántos números desea que sea el vector a utilizar? ";
00032                 cin >> tam;
00033
00034                 vector<int> elementos(tam);
00035
00036                 cout << "\n---> ¿Qué números desea agregar al vector?\n";
00037
00038                 for (int i = 0; i < tam; i++)
00039                 {
00040                     cout << "[Índice " << i << "] " << "Número: ";
00041                     cin >> elementos[i];
00042                 }
00043
00044                 sort(elementos.begin(), elementos.end());
00045
00046                 if (busqueda != nullptr)
00047                 {
00048                     delete busqueda;
00049                 }
00050
00051                 busqueda = new BusquedaBin<int>(elementos);
00052                 break;
00053             }
00054             case 2:
00055             {
00056                 if (busqueda == nullptr)
00057                 {
00058                     cout << "\nNo se ha creado un vector aún.";
00059                     break;
00060                 };
00061
00062                 int num;
00063                 cout << "\nIngrese el número a buscar mediante la versión iterativa: ";
00064                 cin >> num;
00065
00066                 int indice = busqueda->buscaIterativa(num);
00067
00068                 if (indice == -1)
00069                 {
00070                     cout << "\nNo se ha encontrado el número en el vector.";
00071                 }
00072                 else
00073                 {
00074                     cout << "\n;Se ha encontrado el número!";
00075                     cout << "\n---> Índice del número " << num << " : " << indice;
00076                 }
00077                 break;
00078             }
00079             case 3:
00080             {
00081                 if (busqueda == nullptr)
00082                 {
00083                     cout << "\nNo se ha creado un vector aún.";
00084                     break;
00085                 };
00086
00087                 int num;
00088                 cout << "\nIngrese el número a buscar mediante la versión recursiva: ";
00089                 cin >> num;
00090
00091                 int indice = busqueda->buscaRecursiva(num);
00092
00093                 if (indice == -1)
00094                 {
00095                     cout << "\nNo se ha encontrado el número en el vector.";
00096                 }
00097                 else
00098                 {
00099                     cout << "\n;Se ha encontrado el número!";
00100                     cout << "\n---> Índice del número " << num << " : " << indice;
00101                 }
00102                 break;

```



```

00103     }
00104     case 4:
00105     {
00106         cout << "\nSaliendo del programa...";
00107         exit(0);
00108         break;
00109     }
00110     }
00111 }
00112
00113 delete busqueda;
00114 }
00115
00116 /*****
00117
00123
00124 int menu()
00125 {
00126     int opcion;
00127     do
00128     {
00129         cout << "\n\n----- i MENÚ ! -----";
00130         cout << "\n\n-----> ;Búsqueda binaria!\n";
00131         cout << "1) Crear un vector de n elementos.\n";
00132         cout << "2) Buscar un número a partir de la versión iterativa del algoritmo.\n";
00133         cout << "3) Buscar un número a partir de la versión recursiva del algoritmo.\n";
00134         cout << "4) Salir del programa.\n";
00135         cout << "\n--> Seleccione una opción: ";
00136         cin >> opcion;
00137     } while (opcion < 1 || opcion > 4);
00138     return opcion;
00139 }
00140
00141 /*****
00142
00205
00206 /*****
00207
00239
00240 /*****
00241

```


Index

Análisis, [3](#)

Bibliografía, [9](#)

buscalterativa

 BusquedaBin< T >, [20](#)

buscaRecursiva

 BusquedaBin< T >, [20](#)

BusquedaBin

 BusquedaBin< T >, [19](#)

BusquedaBin< T >, [19](#)

 buscalterativa, [20](#)

 buscaRecursiva, [20](#)

 BusquedaBin, [19](#)

Búsqueda Binaria (C++), [1](#)

headers Directory Reference, [17](#)

headers/busqueda_bin.h, [23](#)

main

 main.cpp, [26](#)

main.cpp

 main, [26](#)

 menu, [27](#)

menu

 main.cpp, [27](#)

Pruebas, [5](#)

README.md, [24](#)

src Directory Reference, [17](#)

src/busqueda_bin.cpp, [24](#)

src/main.cpp, [25](#), [27](#)